

Solving the Lorenz-1960 ODE System Using ANN and Identifying the Optimal Architecture

Department of CSE, United International University

FYDP-I research proposal

Problem statement: Can a plain ANN learn the Lorenz-1960 solution accurately, and which ANN architecture works best when compared against reliable numerical and PINN-style references?



Ordinary differential equations describe the evolution of many physical systems, and the Lorenz-1960 model is a compact nonlinear benchmark that is already used in the literature.

ODE Context

ODEs appear in weather dynamics, fluid interaction, control systems, and many other time-evolving physical processes.

Lorenz-1960 Benchmark

The Lorenz-1960 equations are great for testing because they are highly chaotic and unpredictable, making them just hard enough to truly test.

Reference Literature

We are building on top of the latest AI research, like networks that learn entire math operations or continuous models.

Why Numerical Solvers Are Not the Full Story

Classical methods such as Runge-Kutta are accurate, but they solve the system again for each experiment and do not directly produce a learned surrogate.

Why ANN Is Interesting?

A trained ANN can approximate the solution map quickly after training, which is attractive when repeated evaluation or future extension to multiple settings is needed.

Why This Benchmark?

Lorenz-1960 is small enough to study carefully but nonlinear enough to make architecture choice and solver comparison meaningful.

Our motivation is to build a reliable numerical baseline now, then use it to judge which ANN architecture is actually worth trusting later.

1

Problem-Focused Benchmark

A clean benchmark centered on the Lorenz-1960 ODE system rather than a generic ANN demo.

2

Architecture Study Direction

A research path aimed at discovering how depth, width, activation, and training strategy affect solution quality.

3

Comparison Framework

A common evaluation frame for numerical solvers, plain ANN, and PINN-style references on the same problem.

4

Reusable Workflow

A reproducible pipeline for literature review, solver implementation, validation, and later ANN experiments.

The novelty is not the existence of ANN solvers alone. The novelty is the specific benchmark, comparison frame, and architecture question being asked together.

What Existing Work Already Does

Paper 1 solves Lorenz-1960 with DeepONet-style operator learning. Paper 2 reviews ANN-based differential equation methods. Paper 3 introduces Neural ODEs as a different paradigm.

What Is Missing

The literature does not clearly answer which plain ANN architecture is best for solving the Lorenz-1960 benchmark under a unified experimental setup.

Why That Gap Matters

Without this architecture-level evidence, it is hard to justify when a plain ANN is sufficient and when a more structured neural solver is necessary.

Why It Is Complex

The problem combines mathematical modeling, numerical accuracy, software implementation, model evaluation, and design trade-off decisions. There is no single closed-form shortcut that solves all of these at once.

The Lorenz-1960 system is nonlinear and coupled, so small implementation mistakes can propagate across all three states.

A trustworthy ANN study needs a validated numerical reference first, otherwise model claims are weak.

Architecture choice creates a multi-factor design problem involving depth, width, activation, optimizer, and stability.

The final answer must balance accuracy, computational cost, reproducibility, and clarity of comparison.

Engineering complexity = model design + solver validation + evidence-based comparison

What FYDP-I Will Cover

Implement the Lorenz-1960 equations using a Python ODE solver library. Implement a custom Runge-Kutta baseline.

Optimize and validate the numerical reference outputs for later ANN comparison.

Immediate Deliverables

Literature review matrix and gap analysis
Verified numerical solution plots
Reusable solver code and baseline evaluation notes

What Is Not Promised Yet

Final ANN architecture optimization
Full ANN versus PINN experiment set
Final recommendation across all solver families